Unité 6 — Langages sûrs en mémoire (Rust)

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** « Évaluer les bonnes pratiques et mécanismes de défense ». Les unités 2 à 4 ont montré des bugs mémoire et des défenses *a posteriori*. Cette unité présente l'approche *a priori* : un langage **sûr en mémoire par construction**. *Correspondance directe avec meltypot (deck « Software Security » : Rust's bet, ownership, borrowing, lifetimes, cost model, unsafe).*

1. Le pari de Rust

La plupart des failles graves en C/C++ sont des erreurs de sûreté mémoire (Unités 2–3). Le pari de Rust : **garantir à la compilation** l'absence de ces erreurs — sans ramasse-miettes, donc sans coût à l'exécution. Trois idées liées : **propriété (ownership)**, **emprunt (borrowing)**, **durées de vie (lifetimes)**. Elles répondent à la question centrale de l'aliasing (Unité 3) : « qui a le droit de lire/écrire cette case, et quand ? ».

2. Propriété (ownership)

Chaque valeur a un unique **propriétaire** ; quand il sort de portée, la valeur est libérée automatiquement (pas de `free` manuel → pas de double-free, pas d'oubli). Affecter *déplace* la propriété : l'ancien nom devient invalide.

RUST — DÉPLACEMENT DE PROPRIÉTÉ (NE COMPILE PAS)

```
fn main() {
    let s = String::from("secret");
    let t = s;              // ownership deplace vers t
    println!("{}", s);      // ERREUR : s n'est plus valide
}
```

Le compilateur refuse d'utiliser `s` après le déplacement : la classe entière des use-after-free (Unité 3) devient impossible.

3. Emprunt (borrowing)

On peut **emprunter** une valeur par référence sans en prendre la propriété. Règle fondamentale : à tout instant, **soit** plusieurs emprunts immuables (`&`), **soit** un seul emprunt mutable (`&mut`) — jamais les deux. C'est exactement ce qui empêche l'invalidation d'itérateur vue en C++ (Unité 3).

RUST — EMPRUNT IMMUABLE + MUTATION (NE COMPILE PAS)

```
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];      // emprunt immuable
    v.push(4);              // emprunt mutable simultane -> ERREUR
    println!("{}", first);
}
```

4. Durées de vie (lifetimes)

Le compilateur vérifie qu'une référence ne survit jamais à la valeur qu'elle désigne : plus de référence pendouillante (dangling), donc plus de use-after-free par pointeur mort.

RUST — RÉFÉRENCE PENDOUILLANTE (NE COMPILE PAS)

```
fn longest<'a>(x: &'a str) -> &'a str { x }
fn main() {
    let r;
    {
        let s = String::from("temporaire");
        r = longest(&s);    // s ne vit pas assez longtemps -> ERREUR
    }
    println!("{}", r);
}
```

5. Pourquoi ces règles suffisent

Bug C/C++ (Unités 2-3)	Empêché par
Use-after-free	ownership (libération au bon moment) + lifetimes
Double-free	un seul propriétaire
Invalidation d'itérateur	règle d'emprunt (&mut exclusif)
Débordement (hors-bornes)	vérification des bornes (par défaut)
Référence pendouillante	lifetimes

6. L'échappatoire : `unsafe`

Certaines opérations (interfaçage C, abstractions bas niveau) exigent de sortir des garanties du compilateur : elles sont confinées dans des blocs `unsafe`. L'intérêt : le code à auditer de près est **explicitement délimité et minoritaire**, au lieu d'être partout comme en C.

7. Modèle de coût

Critère	C / C++	Rust
Sûreté mémoire	à la charge du développeur	garantie à la compilation
Coût à l'exécution	nul	nul (vérifs à la compilation)
Ramasse-miettes	non	non
Courbe d'apprentissage	—	plus raide (emprunteur)

Rust n'élimine pas les failles *logiques* (injections, mauvaise autorisation) : celles-ci relèvent toujours des Unités 7–10.

8. Points clés

- Ownership + borrowing + lifetimes = sûreté mémoire vérifiée à la compilation, sans coût d'exécution.
- Les bugs exploités en C (Unités 2–3) ne compilent tout simplement pas en Rust sûr.
- `unsafe` délimite le peu de code à auditer manuellement.

- Les failles logiques (injections) restent : la sûreté mémoire n'est pas la sécurité totale.

Correspondance directe avec meltypot ; erreurs du compilateur produites et vérifiées (rustc 1.75).